Lecture 7 revision sheet

Numerical solution of PDEs: finite differences and the finite difference method

This sheet lists the definitions, formulas and results from Lecture 7, without derivations or discussion. These are in the full notes (`pdes1.ipynb` and the homework notebook); the grey pointers give the section. Lecture 7 is the last numerical-methods lecture of the module, so there are no forward tags; references such as (L1) or (L6) point back to where an idea first appeared. [*] marks material that is optional in the notes, and [**] material that is advanced.

Key messages

1. **Finite differences** approximate derivatives by weighted sums of function values on a *stencil*. Taylor series (L1) give their order of accuracy: a bigger stencil allows a higher order, at a higher cost.
2. Too large a Δx gives a large truncation error; too small a Δx is swamped by round-off (L4).
3. The finite difference method turns a BVP into a (sparse) linear system $A\boldsymbol{c} = \boldsymbol{b}$, and an unsteady PDE into a system of ODEs $\mathrm{d}\boldsymbol{c}/\mathrm{d}t = A\boldsymbol{c}$ (the *method of lines*), which any ODE solver (L6) can integrate.
4. For advection, the **central** scheme is second order but produces wiggles if the cell Peclet number $Pe_c > 2$. **Upwind** is always wiggle-free but only first order, and adds numerical diffusion.
5. **Stable is not the same as accurate.** A smooth, plausible-looking solution can be the answer to a different (more diffusive) problem.
6. Explicit time-stepping of advection–diffusion has **three stability numbers**: $Pe_c = U\Delta x/\kappa$, the Courant number $C = U\Delta t/\Delta x$ and $r = \kappa\Delta t/\Delta x^2$. All the limits must hold at once, and the numerics must be chosen with the physics (U, κ) in mind.

1 Finite difference approximations

Full notes: *Finite differences; Taylor series to estimate accuracy; The central difference; Approximating second derivatives*

- Derivative: $f'(x_0) = \lim_{h\to 0} \dfrac{f(x_0+h) - f(x_0)}{h}$. A finite difference replaces the limit by a small but finite spacing $h = \Delta x$.
- **Order of accuracy p:** the error is $\mathcal{O}(\Delta x^p)$, so halving Δx divides the error by about 2^p. Taylor series analysis always leaves an error proportional to an integer power of Δx. A log–log plot of error against Δx has slope p (L1, L4).

Standard stencils (uniform spacing Δx; $f_j := f(x_0 + j\Delta x)$)

Name	Approximation	Leading error (approx. − exact)	Order
Forward	$f'(x_0) \approx \dfrac{f_1 - f_0}{\Delta x}$	$+\dfrac{\Delta x}{2} f''(x_0)$	1
Backward	$f'(x_0) \approx \dfrac{f_0 - f_{-1}}{\Delta x}$	$-\dfrac{\Delta x}{2} f''(x_0)$	1
Central	$f'(x_0) \approx \dfrac{f_1 - f_{-1}}{2\Delta x}$	$+\dfrac{\Delta x^2}{6} f'''(x_0)$	2
Central, 2nd derivative	$f''(x_0) \approx \dfrac{f_1 - 2f_0 + f_{-1}}{\Delta x^2}$	$+\dfrac{\Delta x^2}{12} f''''(x_0)$	2

- The central difference is more accurate because it uses an interval *symmetric* about x_0: the f'' terms cancel.
- The second-derivative stencil is a central difference of the two half-interval differences $f'(x_0 \pm \Delta x/2) \approx (f_{\pm 1} - f_0)/(\pm\Delta x)$. It needs a three-point stencil.
- **How small can Δx go?** Too large: the truncation error makes the result useless (what is "large" depends on the length scales of f). Too small: round-off error dominates (L4). For the central difference the total error (truncation $\mathcal{O}(\Delta x^2)$ plus round-off $\approx \epsilon/\Delta x$) is smallest near $\Delta x \sim \epsilon^{1/3}$: about 10^{-5} in double and 5×10^{-3} in single precision.
- **Larger stencils** [**]: more points give higher orders, e.g. $f'(x_0) \approx \frac{-11f_0+18f_1-9f_2+2f_3}{6\Delta x}$ (third order, one-sided) or $\frac{-f_2+8f_1-8f_{-1}+f_{-2}}{12\Delta x}$ (fourth order, central). For any stencil offsets s_j, the weights a_j for the k-th derivative solve $\sum_j a_j s_j^m = k!\,\delta_{mk}$ for $m = 0, \ldots, n-1$; then $f^{(k)}(x_0) \approx \Delta x^{-k} \sum_j a_j f(x_0 + s_j\Delta x)$ (the *method of undetermined coefficients*). (Homework: larger stencils.)

2 The model problem: advection–diffusion

Full notes: *Introduction to BVPs; Problem non-dimensionalisation; The time-dependent (unsteady) advection-diffusion problem*

$$\frac{\partial c}{\partial t} + U\frac{\partial c}{\partial x} = \kappa\frac{\partial^2 c}{\partial x^2}, \qquad x \in [0, L],\ t \in [0, \infty),$$

$$c(x, 0) = c^0(x) \ \text{(initial condition)}, \qquad c(0, t) = 0, \quad c(L, t) = c_E \ \text{(Dirichlet boundary conditions)}.$$

- c: concentration, *advected* with velocity U and *diffused* with diffusivity κ (units of κ: length^2/time).
- **Counting conditions:** first order in time ⇒ one initial condition; second order in space ⇒ two boundary conditions. Alternatives: far-field conditions (e.g. $c \to 0$ as $x \to \infty$) on an unbounded domain, or periodic boundaries.
- **Non-dimensional form** (hats dropped): $c_t + c_x = \frac{1}{Pe}\, c_{xx}$ on $[0, 1]$ with $c(0) = 0$, $c(1) = 1$, where the **Peclet number** $Pe = UL/\kappa$ measures advection relative to diffusion. It is the only parameter left.
- **Higher dimensions** [*]: conservative form $c_t + \nabla \cdot \boldsymbol{X} = 0$ with flux $\boldsymbol{X} = \boldsymbol{U}c - \kappa\nabla c$ (advective plus diffusive flux; the basis of the finite volume method). If κ is uniform and $\nabla \cdot \boldsymbol{U} = 0$ this is equivalent to the advective form $c_t + \boldsymbol{U} \cdot \nabla c = \kappa\nabla^2 c$.

3 Steady problem: a boundary value problem (BVP)

Full notes: *Model problem – BVP: steady-state advection-diffusion; Methods for solving BVPs; Defining a grid (or mesh); Dealing with Dirichlet boundary conditions; Solving the diffusion equation*

- At steady state $c_t = 0$ and the initial condition no longer matters:

$$\frac{\mathrm{d}c}{\mathrm{d}x} = \frac{1}{Pe}\frac{\mathrm{d}^2 c}{\mathrm{d}x^2}, \quad c(0) = 0,\ c(1) = 1 \qquad \Longrightarrow \qquad c(x) = \frac{e^{Pe\,x} - 1}{e^{Pe} - 1}.$$

  A **BVP** has its conditions at different boundaries (an IVP has them all at one point).
- Small Pe (diffusion dominates): nearly linear. Large Pe (advection dominates): a thin **boundary layer** at $x = 1$, which is hard to resolve on a uniform mesh. Pure diffusion ($U = 0$): c is linear between the boundary values.
- **Solution methods:** the *shooting method* (solve as an IVP from one boundary and adjust the start until the other condition holds: a root-finding problem), or a *global* discretisation solved as one linear system. The lecture uses the second, because it generalises to PDEs.

Grid and boundary conditions (ghost nodes)

- Uniform mesh, N interior nodes plus two **ghost nodes** just outside the domain:

$$x_i = \left(i - \tfrac{1}{2}\right)\Delta x, \quad i = 0, 1, \ldots, N + 1, \qquad \Delta x = L/N.$$

  So $x = 0$ lies midway between x_0 and x_1, and $x = L$ midway between x_N and x_{N+1}.
- Dirichlet conditions by averaging (second-order accurate): $\frac{1}{2}(c_0 + c_1) = 0$ and $\frac{1}{2}(c_N + c_{N+1}) = c_E$. The same interior stencil can then be used at every interior node $i = 1, \ldots, N$.
- Result: a tridiagonal linear system $A\boldsymbol{c} = \boldsymbol{b}$ of size $N + 2$. The first and last rows hold the boundary conditions; interior row i holds the discretised equation. Solve it with `np.linalg.solve`, not by forming A^{-1} (L2).

4 Discretising advection: central vs upwind

Full notes: *The time-independent (steady) advection-diffusion problem; Upwind for advection; Analysis of the upwind scheme [*]; Practicalities!*

Diffusion always uses the central stencil $\kappa\,\frac{c_{i+1} - 2c_i + c_{i-1}}{\Delta x^2}$ (second order). For advection $U\,\partial c/\partial x$ at node i:

Scheme	Advection term	Order	Behaviour
Central	$U\dfrac{c_{i+1} - c_{i-1}}{2\Delta x}$	2	Accurate when resolved, but **wiggles** (over/undershoots, e.g. negative concentrations) when $Pe_c > 2$.
Upwind	$U\dfrac{c_i - c_{i-1}}{\Delta x}$ if $U > 0$; $U\dfrac{c_{i+1} - c_i}{\Delta x}$ if $U < 0$	1	No wiggles at any resolution, but **numerical diffusion** smears sharp features (e.g. thickens the boundary layer).

- **Cell (grid) Peclet number:** $Pe_c := \dfrac{U\Delta x}{\kappa}$. For the central scheme, $Pe_c \le 2$ rules out wiggles; $Pe_c > 2$ allows them (and in the steady 1D model problem they appear). Coarsening the mesh has the same effect as increasing Pe.

- **Upwind** uses information from the direction the flow comes *from* ("look upstream"). Coupled with central diffusion, the overall scheme is still only **first order**: the first-order advection error dominates (homework: upwind vs central).
- **Modified equation** [*]: $U\frac{c_i-c_{i-1}}{\Delta x} = Uc_i' - \frac{U\Delta x}{2}c_i'' + \mathcal{O}(\Delta x^2)$, so upwind solves (to leading order) a problem with extra diffusivity $\kappa_{\text{num}} = \frac{U\Delta x}{2}$, and $\kappa_{\text{num}}/\kappa = \frac{1}{2}Pe_c$. For accuracy we need $Pe_c \ll 1$. At $Pe_c = 2$, numerical and physical diffusion are equal.
- **Fixes for a Pe_c problem:** a finer mesh (cost), a different discretisation such as upwind (accuracy), or changing the physical problem (a modelling decision, not a numerical one).
- **Instability vs inaccuracy:** an unstable solution is obviously wrong; a stable but over-diffused one may look plausible. Only use a stable-but-inaccurate method if you understand (and ideally quantify) its error.

Lecture example ($Pe = 10$, $Pe_c = 10/N$): $N = 32, 16, 8$ ($Pe_c = 0.31, 0.63, 1.25$) give accurate central solutions; $N = 4$ ($Pe_c = 2.5$) gives a negative undershoot of about -0.25 next to the boundary layer.

5 Unsteady problem: the method of lines

Full notes: *The Method of Lines; Discretising in space; The semi-discrete matrix system; Aside: Sparse matrices for efficiency; Periodic boundaries*

- **Method of lines (semi-discretisation):** discretise in space only. Each $c_i(t) \approx c(x_i, t)$ then satisfies an ODE that couples it to its neighbours, giving a large *coupled ODE system* that any time-stepper (L6) or `solve_ivp` can integrate:
$$\frac{\mathrm{d}c_i}{\mathrm{d}t} = \kappa\frac{c_{i+1} - 2c_i + c_{i-1}}{\Delta x^2} - U\frac{c_{i+1} - c_{i-1}}{2\Delta x}, \qquad c_i(0) = c^0(x_i).$$
The spatial truncation error is $\mathcal{O}(\Delta x^2)$.
- **Boundary conditions:** differentiate the ghost-node conditions in time (for constant c_E), e.g. $\frac{1}{2}(\dot{c}_0 + \dot{c}_1) = 0$, with initial values $c_0^0 = -c_1^0$ and $c_{N+1}^0 = 2c_E - c_N^0$. This gives
$$M\frac{\mathrm{d}\boldsymbol{c}}{\mathrm{d}t} = B\boldsymbol{c} \quad\Longrightarrow\quad \frac{\mathrm{d}\boldsymbol{c}}{\mathrm{d}t} = A\boldsymbol{c}, \quad A := M^{-1}B,$$
where M is the identity except for $(\frac{1}{2}, \frac{1}{2})$ in its first and last rows. Interior row i of B multiplies (c_{i-1}, c_i, c_{i+1}) by $(r_{\text{diff}} + r_{\text{adv}},\ -2r_{\text{diff}},\ r_{\text{diff}} - r_{\text{adv}})$, with $r_{\text{diff}} = \kappa/\Delta x^2$ and $r_{\text{adv}} = U/(2\Delta x)$; the first and last rows of B are zero.
- **Sparse matrices:** each row has only a few non-zeros (the stencil). Dense storage costs $\mathcal{O}(N^2)$ memory, sparse (CSR/CSC in `scipy.sparse`) about $\mathcal{O}(N)$, and sparse products and solves are much faster. Use them for anything but small examples.

Periodic boundaries

- What leaves one end re-enters at the other (the mesh is topologically a circle), so a simulation can run indefinitely without boundary effects. No ghost nodes are needed: use $x_i = i\,\Delta x$, $i = 0, \ldots, N-1$, $\Delta x = L/N$ (a node at $x = L$ would duplicate x_0).
- The neighbours of the end nodes wrap around, so the matrices get corner entries:
$$A = -\frac{U}{\Delta x}D_x + \frac{\kappa}{\Delta x^2}D_{xx}, \qquad D_x:\ \text{rows } (-\tfrac{1}{2},\ 0,\ \tfrac{1}{2}), \quad D_{xx}:\ \text{rows } (1,\ -2,\ 1),$$
including the wrapped entries $(D_x)_{0,N-1} = -\frac{1}{2}$, $(D_x)_{N-1,0} = \frac{1}{2}$ and $(D_{xx})_{0,N-1} = (D_{xx})_{N-1,0} = 1$.

6 Discretising in time

Full notes: *Discretising in time; Other time stepping options*

Space–time mesh: x_i as above and $t^n = n\Delta t$; $c_i^n \approx c(x_i, t^n)$ (subscript = space, superscript = time).

Time-stepping the semi-discrete system $\mathrm{d}\boldsymbol{c}/\mathrm{d}t = A\boldsymbol{c}$

FTCS (forward Euler)	$\boldsymbol{c}^{n+1} = (I + \Delta t A)\,\boldsymbol{c}^n$	explicit; matrix–vector product only
BTCS (backward Euler)	$(I - \Delta t A)\,\boldsymbol{c}^{n+1} = \boldsymbol{c}^n$	implicit; one linear solve per step
Crank–Nicolson (trapezoidal)	$(I - \frac{\Delta t}{2}A)\,\boldsymbol{c}^{n+1} = (I + \frac{\Delta t}{2}A)\,\boldsymbol{c}^n$	implicit
Theta method	$\frac{\boldsymbol{c}^{n+1} - \boldsymbol{c}^n}{\Delta t} = A\big(\theta\boldsymbol{c}^{n+1} + (1-\theta)\boldsymbol{c}^n\big)$	$\theta = 0, 1, \frac{1}{2}$ gives the three above

- FTCS pointwise (central advection and diffusion):
$$c_i^{n+1} = c_i^n - \frac{C}{2}\left(c_{i+1}^n - c_{i-1}^n\right) + r\left(c_{i+1}^n - 2c_i^n + c_{i-1}^n\right), \qquad C = \frac{U\Delta t}{\Delta x}, \quad r = \frac{\kappa\Delta t}{\Delta x^2}.$$

- Implicit schemes cost a linear solve per step but are usually much more stable, allowing larger Δt. Accuracy may still limit Δt.
- Only the latest time level needs storing for a two-level scheme; storing all levels is convenient for plots and animations but memory-hungry.
- **Exact solution for testing** (homework): on an infinite (or periodic, if the solution stays away from itself) domain, $c(x,t) = \dfrac{1}{\sqrt{4\pi\kappa t}} \exp\left(-\dfrac{(x-Ut)^2}{4\kappa t}\right)$, a Gaussian that starts as a Dirac delta at $t = 0$; initialise the simulation at some $t > 0$.

7 Stability: three numbers

Full notes: *Stability considerations (a summary); Cell Peclet number; The Courant number and the CFL condition; The r-number/parameter for diffusion; Combining constraints*

Number	Definition	Limit (FTCS)	What it controls
Cell Peclet	$Pe_c = \dfrac{U\Delta x}{\kappa}$	< 2	central advection: wiggles if the mesh cannot resolve sharp features (steady or unsteady)
Courant (CFL)	$C = \dfrac{U\Delta t}{\Delta x}$	< 1	advection: information may travel at most one cell per step
Diffusion r	$r = \dfrac{\kappa\Delta t}{\Delta x^2}$	$< \frac{1}{2}$	diffusion: time step relative to the diffusive time scale $\Delta x^2/\kappa$

They are linked by $C/r = Pe_c$. **All limits must hold at once**; the most restrictive one sets the largest allowed Δt for a given Δx.

- **Origin (L6):** forward Euler is stable when $|1 + \lambda\Delta t| \leq 1$ for every eigenvalue λ of A. The eigenvalues of D_{xx} lie in $[-4, 0]$, which gives $r \leq \frac{1}{2}$ for diffusion.
- **CFL condition** [*]: a scheme can only converge if its numerical *domain of dependence* contains the true one. A three-point explicit stencil propagates information at most $\Delta x/\Delta t$ per unit time, so we need $U < \Delta x/\Delta t$, i.e. $C < 1$. All explicit methods have a limit of the form $C < C_{\max}$.
- **Exact FTCS condition** [*] (from the eigenvalues on a periodic mesh): $C^2 \leq 2r \leq 1$, i.e. $r \leq \frac{1}{2}$ and $C\,Pe_c \leq 2$. So the CFL condition is necessary but not sufficient; the three practical limits above together do guarantee stability. With $\kappa = 0$ (pure advection) FTCS is unstable for every Δt, because central advection has purely imaginary eigenvalues (L6). Forward Euler with *upwind* advection is stable exactly when $C \leq 1$ (homework).
- **Refining:** halving Δx at fixed C means halving Δt; when diffusion dominates (fixed r) it means quartering Δt. Fixing one instability can trigger another, e.g. a finer mesh for Pe_c tightens the CFL and r limits.
- **Fixes for a time-step instability:** a smaller Δt (cost); a time-stepper with a larger stability region, e.g. an implicit or A-stable method (cost per step, and possibly accuracy); a coarser mesh (but check Pe_c and spatial accuracy); or, rarely, changing the physics.

Lecture example (periodic Gaussian, $N = 500$, $\kappa = 0.01$, $U = 1$, $\Delta t = 2 \times 10^{-4}$): $Pe_c = 0.2$, $C = 0.1$ and $r = 0.5$, exactly at the diffusion limit. Increasing Δt to 2.01×10^{-4} gives $r = 0.5025$ and the solution goes unstable.